Metamath Proof Verification in MeTTa: From Pure Prolog to Opaque and Space-Backed PeTTa Verifiers

(AIrxiv Working Paper, 2026-02-25)

Zarathustra Goertzel* Oružji†

February 2026

Abstract

We present **pverify**, a family of Metamath proof verifiers developed in a staged methodology: **(0)** `mmverify` (original PeTTa verifier), **(1)** `pverify_hybrid` (HE MeTTa non-parsing stage based on the `mmverify.py` parsing workflow), **(2)** `pverify.pl` (pure Prolog baseline), **(3)** `pverify_op` (verification semantics in PeTTa/MeTTa with opaque Prolog data structures through FFI), and **(4)** `pverify_op_space` (space-backed storage using specialized `MapSpace`/`SetSpace` backends). All active implementations pass the same 141-test Metamath conformance suite.

On `set.mm` (519K lines, 136,289 statements), controlled solo benchmarks on the same machine yield: 5m 58s for pure Prolog, 28m 31s for `pverify_op`, and 28m 48s for `pverify_op_space`. Thus the space-backed verifier is within 0.95% wall-time of the opaque-state verifier while preserving identical behavior.

Measurement provenance note. Performance/count claims in this draft are snapshot measurements. Primary artifacts: `AIrxiv/airxiv_2026-02-21_pverify_metamath_in_petta.log`, `AIrxiv/airxiv_2026-02-21_pverify_metamath_in_petta.out`, `AIrxiv/airxiv_2026-02-21_pverify_metamath_in_p` plus regression harnesses under `hyperon/PeTTa/tests/test_all_mmverify.sh` and `hyperon/PeTTa/tests/test_all_e`

Methodologically, we make explicit what is and is not in MeTTa. In `pverify_op` and `pverify_op_space`, statement processing, RPN proof checking, substitution, and DV checks are implemented in `.metta`; Prolog is used for parsing and indexed primitives. The specialized spaces are bonafide PeTTa spaces: they support normal space operations (`add-atom`, `remove-atom`, `match`, `get-atoms`) while exposing optimized indexed accessors for verifier workloads. The main engineering conclusion is that FFI granularity, not algorithmic mismatch, remains the dominant optimization axis.

1 Introduction

Metamath [1] is a minimalist proof language whose simplicity—constants, variables, substitution, disjoint-variable constraints, and a stack-based proof checker—makes it an attractive target for implementation in new programming languages.

MeTTa [3] is the primary language of the Hyperon AGI framework. It features non-deterministic evaluation, pattern matching, and a space-based knowledge representation. PeTTa [4] is a Prolog-based implementation of MeTTa that compiles MeTTa programs to SWI-Prolog, providing access to Prolog's mature ecosystem through a foreign function interface (FFI).

*Human supervisor and original mmverify author.

†AI-assisted collaboration (Claude Code, Codex; under human supervision).

This paper describes **pverify**, a family of Metamath proof verifiers developed through human–AI collaboration. The project evolved through several phases:

1. **mmverify** (Goertzel): The original PeTTa Metamath verifier, hand-written, establishing the core verification algorithm in MeTTa with PeTTa-native data structures.
2. **pverify_hybrid** (AI agents): An HE MeTTa non-parsing precursor stage derived from the `mmverify.py` parsing approach.
3. **pverify.pl** (AI, Claude Code): A pure Prolog reimplementation sharing the `mm_primitives.pl` parser.
4. **pverify_op** (AI, Claude Code): The "opaque data structures" architecture—all verification logic in PeTTa, with Prolog providing only $O(\log n)$ AVL trees and ordsets through FFI.
5. **pverify_op_stream** (AI, Claude Code + Codex): A streaming variant of pverify_op that processes statements one-at-a-time from a Prolog-resident token stream, avoiding the need to hold all parsed statements in PeTTa memory simultaneously.
6. **pverify_op_space** (AI, Claude Code + Codex): A space-backed variant of pverify_op that replaces opaque Prolog assocs/ordsets with PeTTa-native `MapSpace` and `SetSpace` backends, achieving full parity while making the verifier's state backend-flexible.

All current pverify variants share a common Prolog parsing layer (`mm_primitives.pl`, 1,224 lines) and are tested against the same 141-test conformance suite covering both positive (correct proofs accepted) and negative (invalid inputs rejected) cases.

1.1 Claims and Scope

What this paper claims.

- Four current `pverify` implementations pass the same 141-test conformance suite.
- Controlled `set.mm` benchmarks are reported for the main methodology chain: `pverify.pl` → `pverify_op` → `pverify_op_space`.
- `mmverify` and `pverify_hybrid` are included as explicit historical/motivational precursors to this chain.
- `pverify_op_space` achieves both conformance parity and near wall-time parity with `pverify_op` (0.95% delta on `set.mm` in current measurements).

What this paper does not claim.

- It is not a mechanized soundness/completeness proof of a verifier; this is addressed in separate Lean work.
- It does not claim per-error-code completeness ("first violated rule implies specific parser code").

2 Architecture

2.1 Methodology Chain and Invariants

The experimental design follows one controlled chain:

1. **Original PeTTa verifier** (`mmverify`): establishes the direct MeTTa implementation style and motivates optimization work.
2. **HE MeTTa precursor** (`pverify_hybrid`): HE MeTTa non-parsing verifier stage based on `mmverify.py` parser conventions.
3. **Pure Prolog baseline** (`pverify.pl`): establishes a native lower-bound implementation over the same parser.
4. **Opaque PeTTa implementation** (`pverify_op`): moves verification semantics into MeTTa, with Prolog used only for parsing and indexed primitives.
5. **Space-backed PeTTa implementation** (`pverify_op_space`): changes storage backend to specialized spaces while preserving verifier semantics.

To keep this comparison honest, each stage preserves:

- the same parser (`mm_primitives.pl`),
- the same acceptance/rejection contract (141-test conformance suite),
- the same benchmark target (`set.mm`).

2.2 Naive MeTTa-Style Baseline (Motivation)

A natural first implementation in MeTTa uses plain lists for maps and sets with linear traversal:

Listing 1: Naive MeTTa-style lookup (linear scan).

```
;;; Labels as list of (pair Key Value)
(= (lookup-label $k $labels)
   (if (== (size-atom $labels) 0)
       not_found
       (let (($h $t) (decons-atom $labels))
         (let (($k0 $v0) $h)
           (if (== $k $k0) $v0 (lookup-label $k $t))))))
```

This style is simple and idiomatic, but it scales poorly when used in hot loops (proof-step dispatch, substitution lookups, DV checks). The methodology chain above is therefore a sequence of controlled moves from naive linear structures to indexed backends while preserving MeTTa-level verifier semantics.

2.3 Shared Prolog Layer

All verifiers in the pverify family share `mm_primitives.pl`, which provides:

- Tokenization of `.mm` files (handling comments, includes, whitespace)
- Recursive-descent parsing into compound terms (`c([...])`, `v([...])`, etc.)

- File-wide semantic validation (duplicate labels, frame balance, typecode consistency, `$d` variable checks, scoping rules)
- Streaming parser (`next_statement/2`) that yields one statement at a time from a token list

2.4 pverify_op: Batch Verification in PeTTa

The key architectural decision in `pverify_op` is a clean separation of concerns:

- **Prolog (pverify_ds.pl, 227 lines):** Provides data structure operations (AVL tree get/put, ordset add/member/union/subtract, compound term manipulation) as FFI exports. These are *opaque* to PeTTa—MeTTa code never inspects the internal structure of an AVL tree or ordset.
- **PeTTa (pverify_op.metta, 693 lines):** Contains *all* verification logic: statement processing (`$c`, `$v`, `$f`, `$e`, `$d`, `$a`, `$p`, scoping), the RPN proof stack machine (`treat-step`), substitution application (`apply-subst`), and disjoint-variable checking (`check-dvs`).

Verification state is threaded functionally as an immutable tuple:

Listing 2: State representation.

```
;;; State = (state Constants FrameStack Labels)
;;;   Constants:  opaque Prolog ordset
;;;   FrameStack: MeTTa list of Frame tuples
;;;   Labels:     opaque Prolog assoc (label -> entry)
```

Each statement processor receives the current state and returns an updated state (or an error), with no mutable global variables.

2.5 pverify_op_stream: Streaming Verification

The streaming variant replaces batch parsing with incremental statement delivery. Three technical challenges arose:

Token list FFI boundary. Metamath token lists contain literal `(` and `)` atoms. PeTTa's FFI layer interprets parentheses as MeTTa grouping, destructively altering the token structure. The solution is to keep the token stream entirely in Prolog memory using `nb_setval/nb_getval` (SWI-Prolog non-backtrackable globals), passing only parsed statement compounds across the FFI boundary.

Listing 3: Streaming helpers in pverify_ds.pl.

```
ds_stream_init(Filename, ok) :-
    ( parse_mm_file_compounds(Filename, _) -> true
    ; throw(mm_error(parse_failed(Filename)))
    ),
    tokenize_mm_file(Filename, Tokens),
    nb_setval(mm_token_stream, Tokens).

ds_stream_next(Result) :-
    nb_getval(mm_token_stream, Tokens),
    next_statement(Tokens, NSResult),
    ( NSResult = [end]    -> Result = end
    ; NSResult = [error]  -> Result = error
    ; NSResult = [Stmt, Rest]
```

```
      -> nb_setval(mm_token_stream, Rest),
         Result = ns(Stmt)
    ).
```

PeTTa accumulator-return compilation. A subtlety in PeTTa's compilation of recursive functions: when a recursive function returns an accumulator variable directly in one branch, the output can be pinned to the initial value. The workaround is to wrap the return value in a tagged constructor:

Listing 4: Streaming loop with tagged return.

```
(= (process-token-stream $state $count)
   (let $result (ds_stream_next)
     (let $tag (ds_stmt_functor $result)
       (case $tag
         ((end
            (let $_ (println! ("Verified" $count "statements."))
              (stream_ok $state)))
          (ns
            (let $stmt (ds_stmt_arg $result 1)
              (pts-continue $stmt $state $count))))))))
```

Semantic validation parity. The streaming parser (`next_statement/2`) performs only syntactic parsing, while the batch parser (`parse_mm_file_compounds/2`) additionally performs 8+ file-wide semantic validations. To maintain parity, `ds_stream_init` performs a full batch parse-and-validate pass first, then tokenizes for streaming. This two-pass approach ensures identical accept/reject behavior while still delivering statements one at a time during verification.

2.6 pverify.pl: Pure Prolog Verifier

For comparison, `pverify.pl` (520 lines) implements the complete verification algorithm in pure SWI-Prolog, sharing only `mm_primitives.pl` for parsing. It uses the same state representation (AVL assoc + ordsets) natively, avoiding all FFI overhead.

2.7 pverify_op_space: Space-Backed Storage

The space-backed variant replaces `pverify_op`'s opaque Prolog data structures with PeTTa-native specialized spaces, moving persistent verifier state closer to idiomatic MeTTa. The design introduces two specialized backends implemented in `lib_zar.pl`:

- **MapSpace**: key–value storage with AVL sidecar index.
- **SetSpace**: membership storage with ordset sidecar index.

Each space provides a dual interface: (1) the normal PeTTa space API (`add-atom`, `match`, `get-atoms`, `remove-atom`) via Prolog `multifile` hooks, and (2) optimized $O(\log n)$ accessors (`sp_map_get`, `sp_set_has`, etc.) that read directly from the sidecar index. Two construction modes are exposed: `canonical` (maintains mirrored atoms) and `index_only` (fast mode, sidecar-only). `pverify_op_space` uses `sp_new_map_fast`/`sp_new_set_fast` for high-throughput verification.

The state representation changes from immutable value threading to mutable space-atom threading at global scope:

Listing 5: Space-backed state representation.

```
;;; State = (state ConstantsSpace FrameStack LabelsSpace)
;;;   ConstantsSpace: SetSpace atom (mutable)
;;;   FrameStack:     MeTTa list of Frame tuples
;;;   LabelsSpace:    MapSpace atom (mutable)
;;;
;;; Frame = (frame Vars DVs FHyps FLabels EHyps)
;;;   Vars/DVs/FLabels use ordset/assoc hot-path structures
```

This hybrid layout is deliberate: `Constants` and `Labels` are persistent global stores and are excellent space candidates; frame-local fields are short-lived and remain on ordset/assoc hot paths for speed. The verifier still walks `FrameStack` functionally in MeTTa:

Listing 6: Frame stack traversal with space-backed fields.

```
(= (empty-frame)
   (let $vars (ds_empty_ordset)
     (let $dvs (ds_empty_ordset)
       (let $fl (ds_empty_assoc)
         (frame $vars $dvs () $fl ()))))

(= (lookup-d-key $key $fs)
   (if (== (size-atom $fs) 0)
       False
       (let ($f $rest) (decons-atom $fs)
         (let (frame $v $dvs $fh $fl $eh) $f
           (if (ds_ord_member $key $dvs)
               True
               (lookup-d-key $key $rest))))))
```

Crucially, this is still bonafide PeTTa verification: the semantic core remains in MeTTa functions (`process-statement`, `treat-step`, `treat-assertion-body`, `check-dvs`, `verify-proof`). Only parser and indexed primitives are delegated to Prolog. Temporary computation (substitution maps, variable collection, DV filtering) continues to use `ds_*` primitives.

This Phase 1 result establishes **backend-flexible verifier state without semantic regression**: 141/141 test parity, identical architecture for all non-storage code. Current full-`set.mm` performance is also near-parity with `pverify_op` (Section 3).

2.8 Why This Is Bonafide MeTTa/PeTTa

For this project, "bonafide MeTTa/PeTTa verifier" means:

1. verifier semantics are in `.metta`,
2. Prolog is a parser/index backend, not the semantic engine,
3. storage uses real PeTTa spaces (with normal space interfaces).

The current implementation satisfies all three criteria. For example, proof-step semantics are dispatched in MeTTa:

Listing 7: Semantic proof-step dispatch remains in MeTTa.

```
(= (treat-step $entry $stack $state)
   (case $entry
```

```
      (((hyp $hkind $stmt) (cons-atom $stmt $stack))
       ((assertion $dvs $fhyps $ehyps $conclusion)
          (treat-assertion-body $dvs $fhyps
            (get-ehyp-stmts-from-pairs $ehyps)
            $conclusion $stack $state)))))
```

Likewise, statement handling remains in MeTTa (`process-statement`, `add-c`, `add-f`, `add-d`); space operations are invoked as backend primitives, not as replacements for verifier control logic.

3 Test Suite and Results

3.1 Conformance Suite

The test suite comprises 141 active test cases, plus 10 harness entries currently skipped because they require features outside verifier scope:

- **Positive tests**: Valid `.mm` files that must be accepted (exit code 0)
- **Negative tests**: Invalid files (syntax errors, duplicate labels, DV violations, proof failures, etc.) that must be rejected (nonzero exit code)

Tests are drawn from:

- `core/small/`: Core Metamath language features
- `unit/`: Focused unit tests for specific edge cases
- `mmverify/`: Tests from the mmverify.py project
- `core/large/`: Large-scale tests including `set.mm`

3.2 Results

Table 1 presents the main methodology-chain benchmark. Each run was executed solo and sequentially on the same machine (ThinkPad, Linux 6.17, SWI-Prolog 9.x), with explicit memory limits (`ulimit -v 6291456`) and one-hour timeout guards. Prolog baseline used `swipl --stack_limit=4g`; PeTTa runs used `run.sh` (`--stack_limit=8g`).

Table 1: Benchmark results on `set.mm` (519K lines, 136,289 statements, February 2026). Solo sequential runs under fixed memory limits.

Verifier	Suite	Wall	CPU	ms/stmt	RSS
pverify.pl	141/141	5m 58s	343s	2.5	5.20 GB
pverify_op	141/141	28m 32s	1703s	12.5	3.00 GB
pverify_op_space	141/141	28m 48s	1719s	12.6	3.00 GB

All values in Tables 1 and 2 are snapshot measurements from the artifacts listed in the Measurement provenance note (§1.1).

To test scaling before full `set.mm`, we also measured `pverify_op` and `pverify_op_space` on progressively larger corpora (Table 2).

On the active small conformance harness, both `pverify_op` and `pverify_op_space` currently report 141/141 with identical 1m 34s runtime.

Table 2: Intermediate scaling checks for opaque vs space-backed PeTTa.

Dataset	Lines	pverify_op (wall)	pverify_op_space (wall)
`hol.mm`	2,251	1.35s	1.24s
`ql.mm`	12,533	6.55s	5.94s
`nf.mm`	61,205	36.50s	34.80s
`set.2010-08-29.mm`	170,876	3m 05.52s	2m 58.87s
`set.mm`	519,291	28m 31.52s	28m 47.77s

4 Performance Analysis

For the primary chain (`pverify.pl` → `pverify_op` → `pverify_op_space`), the key ratios on `set.mm` are:

- pure Prolog vs opaque PeTTa: $28m31.52s/5m58.02s \approx 4.78\times$ wall-time,
- space-backed vs opaque PeTTa: $28m47.77s/28m31.52s \approx 1.0095\times$ wall-time (0.95% delta).

The main result is not that spaces are "faster" yet; it is that state-storage refactoring to spaces can preserve full conformance and near-identical full-corpus performance.

4.1 FFI Call Frequency Is the Dominant Cost

The PeTTa batch verifier performs identical algorithmic work to the pure Prolog verifier—same AVL tree operations, same ordset manipulations, same proof stack machine—but every data structure operation crosses the PeTTa–Prolog FFI boundary. A single `process-statement` call typically invokes 10–30 FFI functions (`ds_assoc_get`, `ds_ord_add`, `ds_stmt_head`, `ds_stmt_tail`, etc.). Over 136,289 statements, this amounts to millions of FFI round-trips.

The per-statement overhead quantifies this directly: 2.5 ms/stmt in pure Prolog vs 12.5 ms/stmt in PeTTa opaque, implying ∼10 ms/stmt overhead dominated by FFI granularity.

4.2 Why Space-Backed Parity Matters

The space-backed variant demonstrates that specialized spaces can be used as first-class verifier storage without destabilizing performance at `set.mm` scale. This gives a clean substrate for next-step optimizations (batched space operations, fewer boundary crossings) while keeping the verifier logic in MeTTa.

4.3 Memory Profile

Interestingly, the PeTTa verifiers use *less* memory (3.0 GB) than pure Prolog (5.2 GB). In PeTTa, opaque Prolog terms are referenced indirectly; in pure Prolog, all intermediate data structures (token lists, parsed statements, verification state) share a single heap, which grows larger under SWI-Prolog's non-compacting GC.

4.4 Benchmark Methodology Note

An earlier benchmark used concurrent long-running verifier processes, which produced severely distorted wall times. All numbers reported here are from solo sequential runs to avoid contention artifacts.

5 Code Size Comparison

Table 3: Lines of code by component.

Component	Lines
mm_primitives.pl (shared parser)	1,224
pverify_ds.pl (FFI data structures)	227
lib_zar.pl (space backends + utils)	405
env_utils.pl (CLI utilities)	39
pverify.pl (pure Prolog verifier)	520
pverify_op.metta (batch PeTTa)	693
pverify_op_space.metta (space-backed)	658
pverify_op_stream.metta (streaming)	703

The space-backed verifier is *shorter* than the batch version (658 vs 693 lines) because mutable space operations eliminate the need to reconstruct state tuples after each mutation—a `sp_map_put` modifies the space in-place and returns `ok`, whereas `ds_assoc_put` returns a new assoc that must be threaded into a fresh state tuple. The space backends themselves (MapSpace + SetSpace + fast/index modes) add substantial shared infrastructure to `lib_zar.pl`, which is shared library infrastructure reusable by any PeTTa program.

The streaming verifier adds only 10 lines over the batch version—the `process-token-stream` loop replaces `process-all-statements`, and the FFI import list gains two entries (`ds_stream_init`, `ds_stream_next`). The Prolog-side streaming support adds ∼30 lines to `pverify_ds.pl`.

6 Lessons Learned

6.1 PeTTa–Prolog FFI Design

The opaque data structure pattern—where PeTTa code never inspects Prolog-side values but manipulates them exclusively through typed FFI calls—proved highly effective for correctness. It provides $O(\log n)$ performance for associative lookups and set operations while keeping the verification logic in idiomatic MeTTa. However, current measurements still show about 4.78× for opaque PeTTa vs pure Prolog on `set.mm`. This indicates that FFI call frequency, not algorithmic complexity, is the main performance bottleneck.

6.2 FFI Boundary Pitfalls

Two non-obvious FFI issues consumed significant debugging time:

1. **Parenthesis atoms**: Prolog atoms `'('` and `')'` are valid Metamath tokens but are interpreted as MeTTa grouping when crossing the FFI boundary. Keeping token lists Prolog-resident was the clean solution.

2. **Boolean representation**: Prolog's `true` and `false` atoms do not map to PeTTa's boolean values. Using `case` pattern matching instead of `if` avoids this mismatch.

6.3 PeTTa Compilation Subtleties

The accumulator-return pinning issue in recursive functions is a PeTTa-specific compilation artifact. The tagged-constructor workaround (`stream_ok $state`) is simple but non-obvious; it was diagnosed through systematic binary-search debugging (progressively simplifying the recursive body until the minimal failing case was isolated) with assistance from Codex.

6.4 Human–AI Collaboration

The pverify family was developed through iterative human–AI collaboration:

- The human author wrote the original mmverify verifier and provided architectural direction, test infrastructure, and quality oversight.
- AI agents (Claude Code, Codex) performed the bulk of implementation, debugging, and optimization work, including the opaque data structure refactoring, streaming architecture, and the systematic debugging of PeTTa compilation issues.
- The 141-test conformance suite served as the shared contract between human and AI: no claims of progress without passing all tests.

7 Future Work: Reducing FFI Overhead

The performance analysis points to a staged optimization strategy. Phase 1 is complete and Phase 2 is partially complete.

Phase 1: Specialized spaces (complete). `pverify_op_space` replaces opaque Prolog data structures with PeTTa-native `MapSpace`/`SetSpace` backends (Section 2.7). This achieves backend-flexible verifier state with 141/141 test parity and no semantic regression (Table 1; provenance note in §1.1). The space backends are generic PeTTa infrastructure (`lib_zar.pl`), not verifier-specific.

Phase 2: Batched hot-path operations. This phase is underway: batched substitutions and variable extraction (`ds_apply_subst`, `ds_find_vars`) are already in use in the current verifier path. Next batched targets are substitution-map construction and repeated label/type lookups in proof loops.

Phase 3: "Many" space APIs. Adding `sp_map_get_many`, `sp_map_put_many`, `sp_set_add_many` would amortize FFI overhead for bulk operations like `foldl-add-c` (adding all constants from a `$c` statement in one call).

Phase 4: Chunked streaming. Replace per-statement `ds_stream_next` with a chunked variant returning K statements per FFI call, reducing streaming FFI calls from 136,289 to $\sim$532 for $K = 256$.

8 Related Work

Metamath verifiers. The canonical reference implementation is `metamath.exe` (C, by Megill). `mmverify.py` (Python, by Heitzig et al.) is a concise ($\sim$300 line) verifier widely used as a starting point. `mm0-rs` (Rust, by Carneiro [2]) focuses on performance. A formally verified Lean 4 verifier exists [5] with a complete soundness-and-completeness proof.

MeTTa implementations. Hyperon Experimental [3] is the primary MeTTa implementation in Rust. PeTTa compiles MeTTa to SWI-Prolog, providing mature Prolog libraries through FFI. Within the implementations and references surveyed for this draft, pverify appears to be the first non-trivial formal verification workload implemented in MeTTa.

9 Conclusion

The pverify family demonstrates that MeTTa, via PeTTa's Prolog backend, is capable of implementing correct Metamath verification at full `set.mm` scale under a controlled methodology chain: `mmverify` → `pverify_hybrid` → `pverify.pl` → `pverify_op` → `pverify_op_space`. All active implementations keep 141/141 conformance in the recorded snapshot measurements (§1.1).

The core result of this paper is methodological and architectural: `pverify_op_space` makes verifier storage space-backed and still stays near performance parity with `pverify_op` (28m 48s vs 28m 32s on `set.mm`, 0.95% wall delta). This demonstrates that the verifier can be made more "space-native" without sacrificing behavior or practical throughput.

Pure Prolog remains significantly faster (5m 58s on `set.mm`); the remaining gap is primarily FFI-boundary granularity rather than algorithmic mismatch. That places the next optimization work on batched boundary crossings, not on rewriting verifier semantics.

References

[1] N. Megill and D. A. Wheeler, *Metamath: A Computer Language for Mathematical Proofs*, Lulu Press, 2019. Available at http://us.metamath.org/downloads/metamath.pdf.

[2] M. Carneiro, "Metamath Zero: Designing a Theorem Prover Prover," *arXiv preprint arXiv:1910.10703*, 2019.

[3] B. Goertzel *et al.*, "Hyperon: A Framework for AGI at the Human Level and Beyond," *arXiv preprint arXiv:2310.18318*, 2023.

[4] D. Vos, "PeTTa: Prolog-based MeTTa Implementation," TrueAGI, 2024.

[5] Z. Goertzel and Oruži, "Verified Verification: A Lean 4 Proof That the Metamath Checker Is Correct," Working paper, February 2026.